

# Critical Security Gates and OpenSSF Scorecard

Enterprise DevSecOps Security Lab

**Document Owner**

Jagriti Banerjee

**Document Type**

Security gate policy and repository posture assessment

**Scope**

CI/CD gates, dependency risk, container risk, IaC risk,  
secret scanning, and OpenSSF Scorecard posture review

Critical Dependency = Fail

Critical Container = Fail

Secret Found = Fail

Critical IaC = Fail

# 1. Executive Summary

This document defines a professional security gate policy for the Enterprise DevSecOps project. The objective is to convert security expectations into measurable, repeatable, and enforceable CI/CD controls. Each gate is designed to stop unsafe code, vulnerable dependencies, leaked secrets, insecure container images, and critical infrastructure-as-code misconfigurations before they progress toward deployment.

The policy follows a strict fail-fast model for the most dangerous findings: critical dependency vulnerabilities, critical container image vulnerabilities, detected secrets, critical Dockerfile misconfigurations, and critical Kubernetes manifest misconfigurations. OpenSSF Scorecard is included as a repository-level security posture check to assess broader supply-chain hygiene, such as token permissions, branch protection, pinned dependencies, dangerous workflows, and repository maintenance.

## Security Gate Lifecycle in Enterprise DevSecOps Pipeline

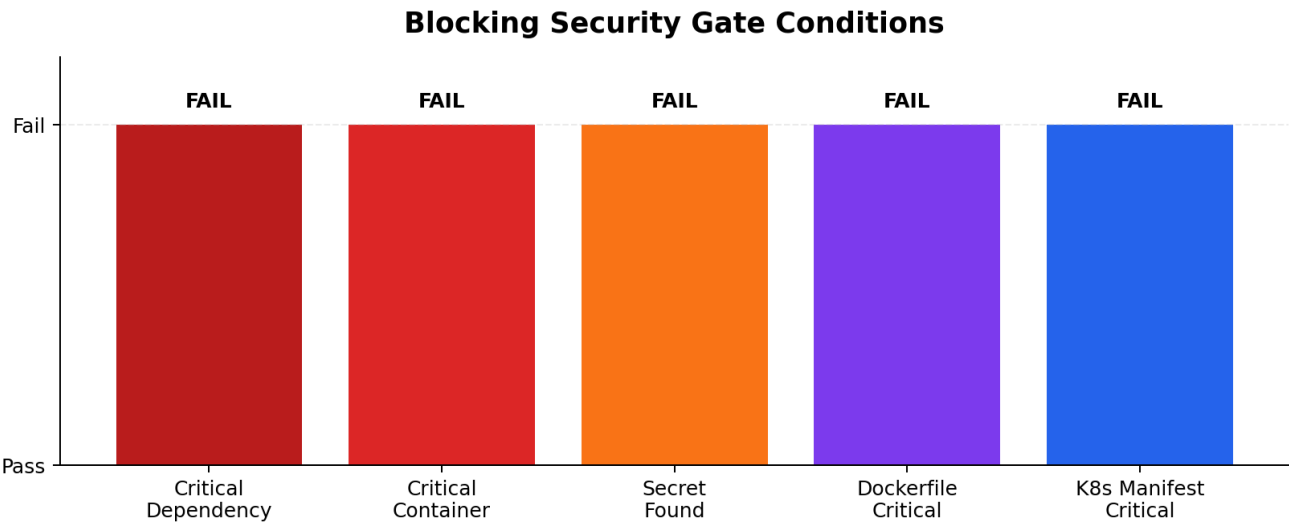


Policy is evaluated on every pull request, push, scheduled run, and manual dispatch.

The project intentionally contains vulnerable application components for lab. defines how a production-grade pipeline should behave after remediation. In must block merges and deployments until validated fixes are committed and

# 2. Gate Policy Summary

| Gate ID | Control Area            | Fail Condition                                         | Primary Tooling                    | Expected Response                                                   |
|---------|-------------------------|--------------------------------------------------------|------------------------------------|---------------------------------------------------------------------|
| GATE-01 | Dependency security     | Critical dependency vulnerability detected             | pip-audit / SCA tooling            | Fail pipeline, open remediation task, upgrade or replace package    |
| GATE-02 | Container security      | Critical container vulnerability detected              | Trivy image scan                   | Fail pipeline, rebuild image with patched base or dependencies      |
| GATE-03 | Secret exposure         | Any secret detected in repository or build context     | Trivy secret scan / secret scanner | Fail pipeline, rotate secret, remove from history, validate cleanup |
| GATE-04 | Dockerfile security     | Critical Dockerfile misconfiguration detected          | Trivy config / IaC scan            | Fail pipeline, harden Dockerfile, retest image build                |
| GATE-05 | Kubernetes IaC security | Critical Kubernetes manifest misconfiguration detected | Trivy config / policy scan         | Fail pipeline, fix manifest, retest with IaC scanner                |
| GATE-06 | Repository posture      | Low or risky Scorecard checks identified               | OpenSSF Scorecard                  | Create backlog item, strengthen repo security posture               |



### 3. Scope and Assumptions

The security gates apply to the Enterprise DevSecOps repository, including the vulnerable Flask application, Dockerfile, dependency manifest, Kubernetes manifests, GitHub Actions workflows, evidence files, and documentation. The gates are designed for pull requests, pushes to protected branches, scheduled scans, and manual security validation runs.

| Area                     | Included in Policy | Reason                                                                                                                   |
|--------------------------|--------------------|--------------------------------------------------------------------------------------------------------------------------|
| Application dependencies | Yes                | Python dependencies can introduce exploitable vulnerabilities even when application code appears safe.                   |
| Container image          | Yes                | Container images include operating system packages, Python libraries, base image layers, and runtime packages.           |
| Secrets                  | Yes                | Secrets in Git can lead to account takeover, registry compromise, cloud compromise, or CI/CD abuse.                      |
| Dockerfile               | Yes                | Dockerfile choices directly impact container privilege, runtime isolation, package exposure, and hardening.              |
| Kubernetes manifests     | Yes                | Kubernetes YAML controls runtime permissions, RBAC, service accounts, network exposure, probes, and resource boundaries. |
| Repository posture       | Yes                | Repository security controls reduce the chance of malicious commits, workflow abuse, and supply-chain compromise.        |

### 4. Security Gate Lifecycle

A security gate is not only a scanner result. It is a decision point. The gate must define what is tested, what threshold is acceptable, what happens on failure, what evidence is captured, and what must be completed before closure. This prevents security checks from becoming passive reports that nobody acts on.

- 1 A developer opens a pull request or pushes code to a protected branch.
- 2 The security-gates workflow runs application, dependency, container, secret, Dockerfile, and Kubernetes manifest scans.
- 3 Each scanner result is converted into a normalized gate decision: pass, fail, warning, or manual review.
- 4 Critical findings fail the pipeline immediately and block unsafe promotion.
- 5 Reports are uploaded as workflow artifacts and, where supported, SARIF results are sent to code scanning.
- 6 The remediation owner fixes the issue, commits the change, and reruns the workflow.
- 7 The gate is closed only after repeat scans confirm the issue is no longer present.

For this project, the critical gate model is intentionally strict because the goal is to show enterprise-grade DevSecOps thinking: critical issues should not rely on manual memory or informal approval. They should be converted into automated stop conditions.

## 5. Detailed Gate Definitions

### 5.1 Critical Dependency Vulnerability = Fail

Severity: Critical

Dependency vulnerabilities are especially dangerous because they can be inherited silently through package versions rather than introduced directly in source code. A developer may not write vulnerable logic, but the deployed application can still be exploitable if a package contains a critical advisory.

| Element                 | Policy Detail                                                                                                                                                                                                                   |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Detection source        | pip-audit scan against src/app/requirements.txt and any lock files or dependency manifests introduced later.                                                                                                                    |
| Fail logic              | If one or more vulnerabilities are classified as critical, the pipeline must fail. High and medium findings may be triaged separately depending on project phase, but critical dependency findings are non-negotiable blockers. |
| Remediation expectation | Upgrade the vulnerable package to a fixed version, replace the dependency, remove unused packages, pin a safe version, and rerun pip-audit until the critical finding is cleared.                                               |
| Evidence required       | pip-audit JSON report, text report, workflow run output, package version diff, and retest result showing no critical findings.                                                                                                  |

### 5.2 Critical Container Vulnerability = Fail

Severity: Critical

Container images combine application code, Python dependencies, operating system packages, runtime libraries, and base image layers. A critical image vulnerability can remain exploitable even if the application code has been fixed.

| Element                 | Policy Detail                                                                                                                                                               |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Detection source        | Trivy image scan after Docker build, using the locally built image or the registry image depending on workflow phase.                                                       |
| Fail logic              | Any critical image vulnerability fails the image gate. High findings may be bounded by a threshold during the lab phase, but critical vulnerabilities should block release. |
| Remediation expectation | Rebuild with a patched base image, upgrade affected packages, remove unnecessary packages, use a slimmer base image, and regenerate the image scan report.                  |
| Evidence required       | Trivy image JSON or SARIF report, Docker image tag, image digest, remediation commit, and clean retest scan.                                                                |

## 5.3 Secret Found = Fail

Severity: Critical

Secrets in repositories or build contexts can allow attackers to access cloud accounts, GitHub packages, container registries, deployment systems, or external services. A discovered secret should be treated as exposed even if the repository is private.

| Element                 | Policy Detail                                                                                                                                                                                                              |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Detection source        | Secret scanning through Trivy filesystem scanning and/or dedicated secret scanning tools in the security-gates workflow.                                                                                                   |
| Fail logic              | Any detected secret fails the workflow. There is no acceptable threshold for live secrets in source control.                                                                                                               |
| Remediation expectation | Remove the secret, rotate it immediately, invalidate old credentials, move sensitive values to GitHub Secrets or a secure vault, and verify the secret is removed from current code and previous history where applicable. |
| Evidence required       | Scanner output, rotation confirmation, commit diff showing removal, and clean retest scan.                                                                                                                                 |

## 5.4 Dockerfile Critical Misconfiguration = Fail

Severity: Critical

Dockerfile misconfigurations can create insecure containers even when application code is secure. Examples include running as root, unsafe package installation patterns, missing hardening controls, excessive privileges, or insecure runtime assumptions.

| Element                 | Policy Detail                                                                                                                                                                              |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Detection source        | Trivy config scan against the Dockerfile and build context configuration.                                                                                                                  |
| Fail logic              | Any critical Dockerfile misconfiguration fails the workflow because it can weaken container isolation or increase runtime blast radius.                                                    |
| Remediation expectation | Apply secure Dockerfile patterns: non-root user, fixed UID/GID, minimal base image, no unnecessary packages, no secrets in layers, safe healthcheck, and least privilege runtime settings. |
| Evidence required       | Trivy config report, Dockerfile diff, rebuilt image evidence, and retest scan confirming the critical issue is resolved.                                                                   |

## 5.5 Kubernetes Manifest Critical Misconfiguration = Fail

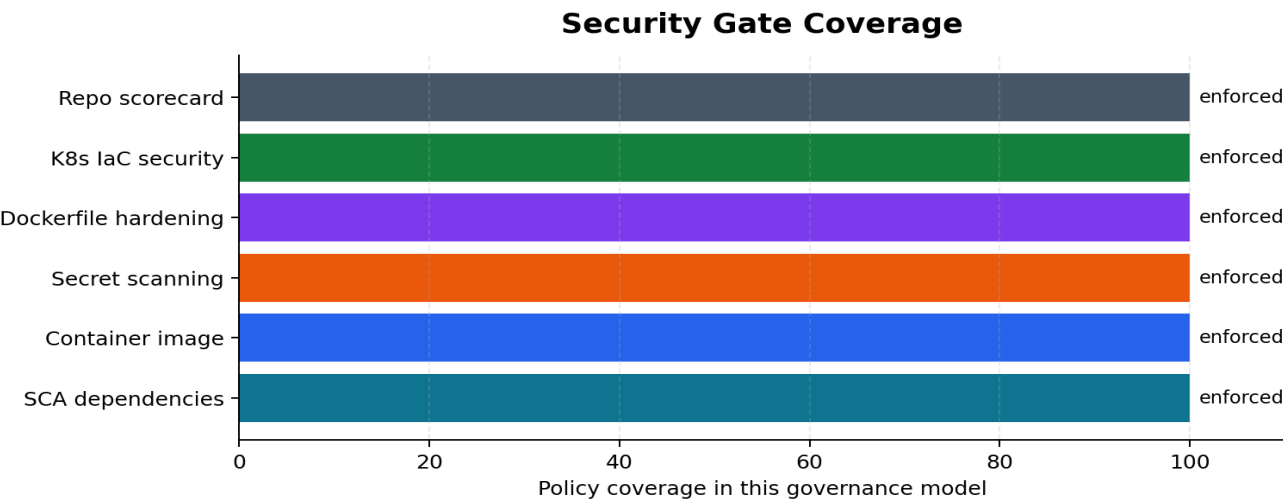
Severity: Critical

Kubernetes manifests define runtime identity, privilege, service exposure, network policy, resource boundaries, and security context. A critical misconfiguration can lead to privilege escalation, workload compromise, or cluster-level risk.

| Element                 | Policy Detail                                                                                                                                                                                                                     |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Detection source        | Trivy config scan against k8s/base and future overlays such as k8s/overlays/dev and k8s/overlays/prod.                                                                                                                            |
| Fail logic              | Any critical Kubernetes manifest misconfiguration fails the workflow. The manifest must not be deployed until corrected and retested.                                                                                             |
| Remediation expectation | Harden the manifest using runAsNonRoot, allowPrivilegeEscalation false, readOnlyRootFilesystem, dropped capabilities, RuntimeDefault seccomp, resource limits, dedicated ServiceAccount, RBAC least privilege, and NetworkPolicy. |
| Evidence required       | IaC scan report, manifest diff, kubectl validation output, and retest scan confirming no critical misconfiguration remains.                                                                                                       |

## 6. OpenSSF Scorecard Role in This Project

OpenSSF Scorecard is useful in this project because it automatically assesses repository security checks and scores the project, helping identify areas to improve in repository security posture. While the security-gates workflow focuses on specific blocking conditions, Scorecard evaluates broader supply-chain hygiene around repository configuration and development workflow security.



Scorecard should not be treated as a replacement for SAST, SCA, container scanning, secret scanning, or Kubernetes IaC scanning. It is a complementary governance control that highlights weaknesses in repository security practices that may not appear as application vulnerabilities.

| Scorecard Area            | Why It Matters                                                                  | Recommended Project Action                                                                            |
|---------------------------|---------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Token permissions         | Overly broad workflow permissions can allow abuse if a workflow is compromised. | Use least-privilege permissions per workflow and avoid write-all defaults.                            |
| Dangerous workflows       | Unsafe workflow patterns can expose secrets or enable script injection.         | Review <code>pull_request_target</code> , untrusted input handling, and shell execution in workflows. |
| Pinned dependencies       | Unpinned actions or dependencies can be changed upstream.                       | Pin critical third-party GitHub Actions to commit SHA in production-grade workflows.                  |
| Branch protection         | Protected branches reduce unauthorized direct changes to main.                  | Require pull requests, reviews, and passing security gates before merge.                              |
| Security policy           | A clear policy improves vulnerability reporting and maintenance maturity.       | Add <code>SECURITY.md</code> with reporting expectations and supported scope.                         |
| Dependency update tooling | Outdated dependencies increase exposure time.                                   | Enable Dependabot or equivalent update automation.                                                    |

## 7. CI/CD Implementation Model

The project should include two separate workflow files: `security-gates.yml` for blocking security controls and `scorecard.yml` for repository posture assessment. Keeping them separate makes reporting clearer and allows repository posture checks to run on a schedule without mixing them with every application security gate decision.

| Workflow           | Purpose                                                                                                                   | Recommended Trigger                                                                          | Blocking Role                                                                      |
|--------------------|---------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| security-gates.yml | Runs SAST, dependency scan, secret scan, Trivy filesystem scan, Trivy image scan, and Kubernetes/Dockerfile config scans. | Pull request, push, manual dispatch, scheduled weekly scan.                                  | Blocking check for protected branches once remediation is complete.                |
| scorecard.yml      | Runs OpenSSF Scorecard and uploads SARIF where supported.                                                                 | Push to main, pull request, branch protection event, manual dispatch, scheduled weekly scan. | Posture check and governance signal; may become required after baseline hardening. |

The security gate workflow should upload reports as artifacts and summarize gate decisions in the GitHub Actions job summary. This makes the workflow useful for recruiters and reviewers because they can see the control logic, the evidence, and the reason for pass/fail decisions without manually opening raw JSON files.

## 8. Policy-as-Code Thresholds

Security gate thresholds should be explicit and visible in the workflow. For this project, the critical thresholds are intentionally zero-tolerance. This means the pipeline is allowed to proceed only if the security control does not detect a critical issue in that category.

| Policy Variable          | Recommended Value | Meaning                                                            |
|--------------------------|-------------------|--------------------------------------------------------------------|
| MAX_PIP_AUDIT_CRITICAL   | 0                 | No critical dependency vulnerability is allowed.                   |
| MAX_TRIVY_IMAGE_CRITICAL | 0                 | No critical container image vulnerability is allowed.              |
| MAX_SECRET_FINDINGS      | 0                 | No secret finding is allowed.                                      |
| MAX_DOCKERFILE_CRITICAL  | 0                 | No critical Dockerfile misconfiguration is allowed.                |
| MAX_K8S_CONFIG_CRITICAL  | 0                 | No critical Kubernetes manifest misconfiguration is allowed.       |
| SCORECARD_MINIMUM_TARGET | 7.0 or higher     | Suggested target for repository maturity; may be raised over time. |

The Scorecard threshold is treated differently from the critical security gates. A low Scorecard result should create remediation work and improvement tasks. After the repository is mature, the team may choose to enforce a minimum Scorecard target as a required check.

## 9. Suggested Workflow Logic

The following pseudo-logic describes how the security-gates workflow should decide whether to fail. The same decision model can be implemented using Python parsing steps, jq filters, SARIF analysis, or native scanner exit codes.

```

dependency_critical_count = count_critical(pip_audit_report)
container_critical_count = count_critical(trivy_image_report)
secret_count = count_secrets(trivy_fs_report)
dockerfile_critical_count = count_critical_dockerfile_misconfigs(trivy_config_report)
k8s_critical_count = count_critical_k8s_misconfigs(trivy_config_report)

if dependency_critical_count > 0:
    fail("Critical dependency vulnerability detected")

if container_critical_count > 0:
    fail("Critical container vulnerability detected")

if secret_count > 0:
    fail("Secret detected in repository or build context")

if dockerfile_critical_count > 0:
    fail("Critical Dockerfile misconfiguration detected")

if k8s_critical_count > 0:
    fail("Critical Kubernetes manifest misconfiguration detected")

pass("All blocking security gates passed")

```

## 10. Evidence Requirements

Every failed gate must produce enough evidence for a reviewer to understand what failed, where it failed, and how to reproduce the result. Evidence should be saved as GitHub Actions artifacts and, where supported, uploaded as SARIF so findings appear in the Security tab.

| Gate                     | Minimum Evidence Artifact                                                  | Preferred Evidence Format |
|--------------------------|----------------------------------------------------------------------------|---------------------------|
| Dependency vulnerability | pip-audit report with package, version, vulnerability ID, and fix version. | JSON plus text summary.   |

| Gate                        | Minimum Evidence Artifact                                                                       | Preferred Evidence Format        |
|-----------------------------|-------------------------------------------------------------------------------------------------|----------------------------------|
| Container vulnerability     | Trivy image report with target image name, severity, package, installed version, fixed version. | JSON plus SARIF.                 |
| Secret found                | Secret scanner report showing rule ID, location, and masked secret.                             | JSON plus redacted text summary. |
| Dockerfile misconfiguration | Trivy config report identifying Dockerfile rule, severity, and affected instruction.            | JSON plus SARIF.                 |
| Kubernetes misconfiguration | IaC scan report identifying YAML file, resource, line, and rule.                                | JSON plus SARIF.                 |
| OpenSSF Scorecard           | Scorecard SARIF and job summary showing evaluated checks.                                       | SARIF plus Actions summary.      |

## 11. Failure Handling Workflow

A failed gate should trigger a consistent remediation process. This is what makes the project look enterprise-ready: the pipeline does not only fail, it produces evidence, directs remediation, and defines how the issue is closed.

- 1 Mark the workflow run as failed and preserve scanner artifacts.
- 2 Create a remediation task describing the affected component, severity, owner, and expected fix.
- 3 For secrets, rotate credentials before considering code cleanup complete.
- 4 For dependencies or images, upgrade to fixed versions and rebuild the affected artifact.
- 5 For Dockerfile or Kubernetes misconfigurations, update IaC and rerun local validation before committing.
- 6 Commit the fix with a clear message referencing the finding or remediation task.
- 7 Rerun the security-gates workflow and attach the clean retest evidence.
- 8 Close the issue only after the gate passes and the remediation reviewer confirms the fix.

For portfolio presentation, each failure and retest should be documented as a before-and-after security story. This helps reviewers understand not only that tools were run, but that findings were interpreted and remediated professionally.

## 12. Branch Protection Recommendations

Security gates become more valuable when they are connected to branch protection. If the gates run but the repository still allows direct pushes to main, the pipeline becomes advisory rather than protective. The following settings are recommended for the Enterprise DevSecOps project after remediation is complete.

| Branch Protection Control         | Recommended Setting                             | Security Value                                            |
|-----------------------------------|-------------------------------------------------|-----------------------------------------------------------|
| Require pull request before merge | Enabled                                         | Prevents direct unreviewed changes to protected branches. |
| Required status checks            | security-gates.yml and optionally scorecard.yml | Prevents merge when blocking security checks fail.        |
| Require review approvals          | At least one reviewer                           | Improves change accountability.                           |
| Require conversation resolution   | Enabled                                         | Ensures security review comments are addressed.           |
| Restrict force pushes             | Disabled for protected branches                 | Prevents destructive history rewriting.                   |
| Require signed commits            | Recommended                                     | Improves commit authenticity where feasible.              |

## 13. Scorecard Improvement Backlog

OpenSSF Scorecard findings should be converted into a repository hardening backlog. This makes the Scorecard result actionable instead of decorative. The backlog below can be used to improve the project over time.

| Backlog Item                                                 | Priority | Expected Outcome                                                          |
|--------------------------------------------------------------|----------|---------------------------------------------------------------------------|
| Add SECURITY.md                                              | High     | Defines vulnerability reporting expectations and project security policy. |
| Pin important third-party actions to SHA                     | High     | Reduces risk of upstream action compromise or unexpected behavior.        |
| Use least-privilege workflow permissions                     | High     | Limits blast radius if a workflow step is abused.                         |
| Enable Dependabot for GitHub Actions and Python dependencies | Medium   | Reduces dependency exposure time.                                         |
| Require security-gates workflow as protected branch check    | High     | Blocks unsafe merges automatically.                                       |
| Publish Scorecard run summary as artifact                    | Medium   | Creates governance evidence for reviewers.                                |
| Review dangerous workflow patterns                           | High     | Reduces risks from untrusted pull request inputs.                         |

## 14. Metrics and Reporting

A mature security gate program should be measured. Metrics show whether the project is improving and whether security issues are being resolved faster over time.

| Metric                 | Definition                                        | Why It Matters                                           |
|------------------------|---------------------------------------------------|----------------------------------------------------------|
| Gate pass rate         | Percentage of runs where all blocking gates pass. | Shows whether code is becoming more secure before merge. |
| Critical finding count | Total critical findings by category.              | Measures immediate release-blocking risk.                |
| Mean time to remediate | Time from failed gate to clean retest.            | Measures remediation speed and operational maturity.     |
| Secrets detected       | Number of secret findings per run.                | Tracks whether secret handling is improving.             |
| Scorecard trend        | Scorecard posture over time.                      | Shows repository security hygiene improvement.           |
| False positive rate    | Findings closed as not applicable.                | Helps tune gates without weakening critical controls.    |

## 15. Retest and Closure Criteria

A gate failure should not be closed because a code change was made. It should be closed only when retest evidence proves that the same detection no longer appears. This document uses the following closure criteria.

| Gate                                          | Closure Criteria                                                                    |
|-----------------------------------------------|-------------------------------------------------------------------------------------|
| Critical dependency vulnerability             | pip-audit shows zero critical dependency vulnerabilities for the affected manifest. |
| Critical container vulnerability              | Trivy image scan shows zero critical vulnerabilities for the rebuilt image.         |
| Secret found                                  | Secret scanner shows zero secrets; exposed secret has been rotated or revoked.      |
| Dockerfile critical misconfiguration          | Trivy config scan shows zero critical Dockerfile misconfigurations.                 |
| Kubernetes manifest critical misconfiguration | Trivy config scan shows zero critical Kubernetes manifest misconfigurations.        |
| Scorecard posture issue                       | Scorecard check improves or documented mitigation/backlog item exists.              |

## 16. Project-Specific Value for Recruiters and Reviewers

This security gate model demonstrates practical DevSecOps maturity. It shows that the project is not simply a vulnerable Flask app or a collection of tools. It connects findings to automated controls, automated failure criteria, remediation workflow, and retest evidence.

- Shows understanding of how CI/CD pipelines prevent unsafe releases.
- Demonstrates strict zero-tolerance policy for critical dependency, container, secret, Dockerfile, and Kubernetes manifest issues.
- Connects scanner output to actionable gate decisions instead of passive reporting.
- Includes OpenSSF Scorecard for repository and supply-chain posture assessment.
- Supports professional evidence generation through artifacts, SARIF, job summaries, and retest reports.
- Provides a realistic enterprise workflow that can be discussed in interviews for DevSecOps, cloud security, application security, and Kubernetes security roles.

## 17. Appendix A - Recommended Artifact Names

| Artifact                               | Purpose                                                          |
|----------------------------------------|------------------------------------------------------------------|
| bandit-security-gate-reports           | SAST evidence and SAST gate decision.                            |
| pip-audit-security-gate-reports        | Dependency vulnerability evidence and SCA gate decision.         |
| trivy-filesystem-security-gate-reports | Repository, secret, vulnerability, and filesystem scan evidence. |
| trivy-config-security-gate-reports     | Dockerfile and Kubernetes manifest misconfiguration evidence.    |
| trivy-image-security-gate-reports      | Container image vulnerability evidence.                          |
| security-gates-final-summary           | Human-readable gate summary across all checks.                   |
| openssf-scorecard-results              | Repository posture Scorecard results in SARIF format.            |

## 18. Appendix B - Interview Explanation

A concise interview explanation for this control model is provided below.

I implemented security gates in the DevSecOps pipeline so that critical dependencies, vulnerabilities, secrets, critical Dockerfile misconfigurations, and critical Kubernetes manifest issues are caught in the CI/CD workflow automatically. I also added OpenSSF Scorecard to evaluate repository posture, including permissions, pinned actions, branch protection, and dangerous workflow permissions. I integrated the OpenSSF Scorecard checklist into automated policy enforcement with evidence artifacts and retest reports.

End of document.